Reviewer Pack — Tropical Geometry of Convolutional Neural Networks vs SAT Sa…

Entry 56aff55f5100 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 04:29:46 UTC

Tropical Geometry of Convolutional Neural Networks vs SAT Satisfiability Time

Entry ID: `56aff55f5100` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 04:29:46 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: SAT Satisfiability Time

Statement:

> {'text': 'For a given Boolean circuit C representing a CNF formula F, the tropical rank of the polynomial associated with the tensor product of C and its dual (C^⊤), denoted as τ(C ⊗ C^⊤), is upper-bounded by a function θ(n) that depends on the size n of the formula. Specifically, τ(C ⊗ C^⊤) ≤ θ(n), where θ(n) = Θ(log n).', 'counterexample': 'For any CNF formula F with clauses not linearly dependent over the tropical semiring, if there exists a Boolean circuit C representing F such that τ(C ⊗ C^⊤) > θ(n), then the conjecture is falsified.'}

Rationale (proposer's reasoning):

> {'text': 'Tropical geometry has shown potential in characterizing complexity classes by studying geometric properties of polynomials. If this relationship between tropical rank and circuit tensor products holds, it could provide a new perspective on the structure of SAT instances and potentially lead to improved algorithms for solving them.'}

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3ec5c61a0dd89fd0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all CNF formulas F, the tropical rank τ(C ⊗ C^⊤) of the tensor product is less than or equal to θ(n) with a margin of error ≤ 3 across at least 25 out of 30 random seeds. The conjecture is falsified if any seed produces a tropical rank greater than θ(n) by more than 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND convolutional neural networks AND SAT satisfiability time` - `tensor product tropical rank CNF formula AND complexity theory` - `Boolean circuit dual polynomial associated upper bound θ(log n)`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1805.07091v1] Tropical Geometry of Deep Neural Networks - [http://arxiv.org/abs/1903.06254v1] Demonstration of Vector Flow Imaging using Convolutional Neural Networks - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1503.00822v2] Product Ranks of the 3×3 Determinant and Permanent - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for k in range(i+1, n):
            factor = Fraction(matrix[k][i], matrix[i][i])
            for j in range(n):
                matrix[k][j] -= factor * matrix[i][j]
```

```python
def determinant(matrix):
    n = len(matrix)
    det = 1
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(ma

    gaussian_elimination(augmented_matrix)

    for i in range(n):
        det *= augmented_matrix[i][n]

    return det

def tensor_product(cnf):
    n = len(cnf)
    m = 2 ** n
    result = [[0] * (m * m) for _ in range(m * m)]

    for i in range(m):
        for j in range(m):
            for k in range(n):
                if (i >> k) & 1:
                    result[i * m + j][(k * m) + j] = -1
                else:
                    result[i * m + j][(k * m) + i] = -1

    return result

def tropical_rank(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i+1, n):
            if matrix[i][j] > matrix[j][i]:
                matrix[i], matrix[j] = matrix[j], matrix[i]

    rank = 0
    for row in matrix:
        if any(x != 0 for x in row):
            rank += 1

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    tensor_prod = tensor_product(cnf)
    rank = tropical_rank(tensor_prod)

    theta_n = math.log(n, 2)
    metric_value = abs(rank - theta_n)
    conjecture_holds = metric_value <= 3
    counterexample = "" if conjecture_holds else f"rank={rank}, theta_n={theta_n}"

    return {
```

```
        "metric_name": "tropical_rank_difference",
        "metric_value": metric_value,
        "instances_tested": n * n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62196894.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62196894.py", line 80, in run_trial
    tensor_prod = tensor_product(cnf)
                  ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62196894.py", line 49, in tensor_produc
    result = [[0] * (m * m) for _ in range(m * m)]
              ~~~~^~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that it was unable to complete its execution and provide a result. | next: Re-run the test with increased memory allocation or optimize the code to avoid the MemoryError.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12871
2	propose	ollama_remote	glm4:latest	0	0	11021
3	preregistration	ollama_remote	glm4:latest	0	0	6240
4	novelty	ollama_remote	glm4:latest	0	0	4592
5	novelty	ollama_remote	glm4:latest	0	0	6559
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15799
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10538
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12501
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11580
10	judge	ollama_remote	glm4:latest	0	0	8033

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99734 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/56aff55f5100.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/56aff55f5100.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/56aff55f5100.tar.gz` (if generated)